

CS 302.1 - Automata Theory

Lecture 06

Shantanav Chakraborty

Center for Quantum Science and Technology (CQST)

Center for Security, Theory and Algorithms (CSTAR)

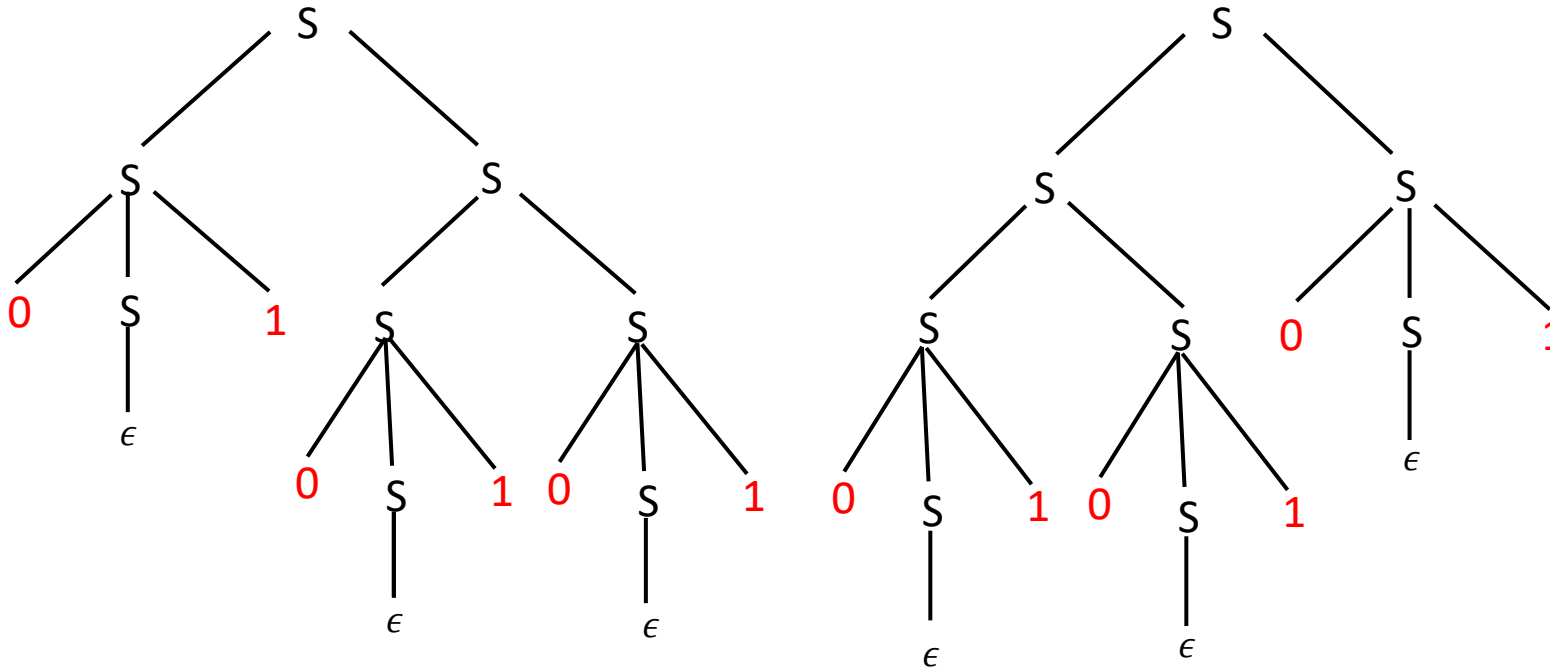
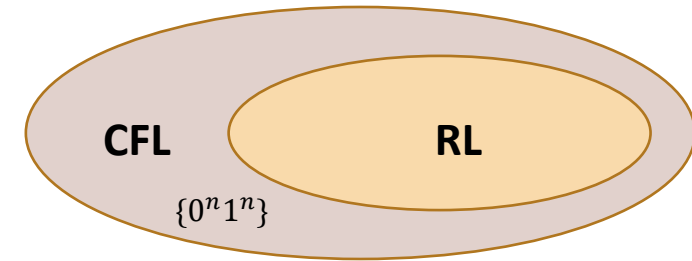
IIIT Hyderabad



Quick Recap

Context-Free Grammars: If the *rules* of the underlying grammar G are of the form
$$V \rightarrow (VUT)^*$$

then such a grammar is called **Context-Free**.



Parse trees: These are ordered trees that provide alternative representations of the derivation of a grammar.

Ambiguous grammars: There exists $\omega \in L(G)$, such that there are **two or more leftmost derivations** for ω (or equivalently two or more rightmost derivations) or equivalently **two or more parse trees** for ω . **Ambiguity** may not be desirable

Chomsky Normal Form

Often it is easier to work with CFG in a simple standardized form - the Chomsky Normal Form (CNF) is one of them.

Chomsky Normal Form

A CFG G is in CNF if every rule of G is of the form

$$Var \rightarrow Var Var$$

$$Var \rightarrow ter$$

$$Start Var \rightarrow \epsilon$$

where Var can be any variable, including the Start Variable, $Start Var$.

Chomsky Normal Form

Often it is easier to work with CFG in a simple standardized form - the Chomsky Normal Form (CNF) is one of them.

Chomsky Normal Form

A CFG G is in CNF if every rule of G is of the form

$$Var \rightarrow Var Var$$

$$Var \rightarrow ter$$

$$Start Var \rightarrow \epsilon$$

where Var can be any variable, including the Start Variable, $Start Var$.

Why are CNFs useful?

- Suppose you are given a CFG G and a string w as input and you have to write an algorithm that decides whether G generates w .
- Your algorithm outputs YES if G generates w and NO, otherwise.

Chomsky Normal Form

A CFG G is in **CNF** if every rule of G is of the form

$$Var \rightarrow Var \ Var$$

$$Var \rightarrow ter$$

$$Start \ Var \rightarrow \epsilon$$

where Var can be any variable, including the Start Variable, $Start \ Var$.

Why are CNFs useful?

- Suppose you are given a CFG G as and a string w as input and you have to **write an algorithm that decides whether G generates w .**
- The algorithm outputs YES if G generates w and NO, otherwise.
 - ❖ One idea is to go through ALL derivations one by one and output YES if any of them generates w .
 - ❖ However, infinitely many derivations may have to be tried.
 - ❖ So if G does not generate w , the algorithm will never stop.
 - ❖ So this problem appears to be **undecidable**.

Chomsky Normal Form

A CFG G is in **CNF** if every rule of G is of the form

$$\begin{aligned}Var &\rightarrow Var\ Var \\Var &\rightarrow ter \\Start\ Var &\rightarrow \epsilon\end{aligned}$$

where Var can be any variable, including the Start Variable, $Start\ Var$.

Why are CNFs useful?

Suppose you are given a CFG G and a string w as input and you have to **write an algorithm that decides whether G generates w** . This problem appears to be **undecidable**.

Chomsky Normal Form

A CFG G is in **CNF** if every rule of G is of the form

$$Var \rightarrow Var Var$$

$$Var \rightarrow ter$$

$$Start Var \rightarrow \epsilon$$

where Var can be any variable, including the Start Variable, $Start Var$.

Why are CNFs useful?

Suppose you are given a CFG G as and a string w as input and you have to **write an algorithm that decides whether G generates w .**

- Converting G first to a CNF alleviates this and **makes the problem decidable**.
- It limits the number of steps in derivations required to generate any $w \in L(G)$.
- If $w \in L(G)$, then a CFG in Chomsky Normal Form has **derivations of $2n - 1$ steps** for input strings w of length n (We will prove this shortly).

Chomsky Normal Form

A CFG G is in **CNF** if every rule of G is of the form

$$\begin{aligned}Var &\rightarrow Var\ Var \\Var &\rightarrow ter \\Start\ Var &\rightarrow \epsilon\end{aligned}$$

where Var can be any variable, including the Start Variable, $Start\ Var$.

A CFG in Chomsky Normal Form has derivations of $2n - 1$ steps for generating strings $w \in L(G)$ of length n .

Why are CNFs useful?

Suppose you are given a CFG G and a string w as input and you have to **write an algorithm that decides whether G generates w .**

1. Convert G to CNF.
2. List all derivations of $2n - 1$ steps, where $|w| = n$. (There are a finite number of these)
3. If ANY of these derivations generate w , output YES, otherwise output NO.

Chomsky Normal Form

A CFG G is in **CNF** if every rule of G is of the form

$$\begin{aligned}Var &\rightarrow Var\ Var \\Var &\rightarrow ter \\Start\ Var &\rightarrow \epsilon\end{aligned}$$

where Var can be any variable, including the Start Variable, $Start\ Var$.

- 1) **A CFG in Chomsky Normal Form has derivations of $2n - 1$ steps for generating strings $w \in L(G)$ of length n .**
- 2) Any CFL can be generated by a CFG written in Chomsky Normal Form.

The proof of 1) is quite simple!

Chomsky Normal Form

Prove that a **CFG in Chomsky Normal Form** has derivations of $2n - 1$ steps for generating strings $w \in L(G)$ of length n .

Proof: Note that any CFG in CNF can be written as:

$$\begin{array}{ll} A \rightarrow BC & [B, C \text{ are not start variables}] \\ A \rightarrow a & [a \text{ is a terminal}] \\ S \rightarrow \epsilon & [S \text{ is the Start Variable}] \end{array}$$

Think of a parse tree that prepares a string w such that $|w| = n$ using the aforementioned rules.

Chomsky Normal Form

Prove that a **CFG in Chomsky Normal Form** has derivations of $2n - 1$ steps for generating strings $w \in L(G)$ of length n .

Proof: Note that any CFG in CNF can be written as:

$$\begin{array}{ll} A \rightarrow BC & [B, C \text{ are not start variables}] \\ A \rightarrow a & [a \text{ is a terminal}] \\ S \rightarrow \epsilon & [S \text{ is the Start Variable}] \end{array}$$

Think of a parse tree that prepares a string w such that $|w| = n$.

Each of the terminals must have come from a rule of the form $A \rightarrow a$, one for each symbol of w . There has to be n such rules.

Chomsky Normal Form

Prove that a **CFG in Chomsky Normal Form** has derivations of $2n - 1$ steps for generating strings $w \in L(G)$ of length n .

Proof: Note that any CFG in CNF can be written as:

$$\begin{array}{ll} A \rightarrow BC & [B, C \text{ are not start variables}] \\ A \rightarrow a & [a \text{ is a terminal}] \\ S \rightarrow \epsilon & [S \text{ is the Start Variable}] \end{array}$$

Think of a parse tree that prepares a string w such that $|w| = n$.

Each of the terminals must have come from a rule of the form $A \rightarrow a$, one for each symbol of w . There has to be n such rules.

Also, before producing these terminals, the grammar must create n variables from the single initial variable S , using rules such as $A \rightarrow BC$.

Each such rule increases the number of variables by 1. So, there has to be $n - 1$ such rules in all to create the n variables.

So, total number of rules to derive w is $n - 1 + n = 2n - 1$.

Chomsky Normal Form

A CFG G is in **CNF** if every rule of G is of the form

$$\begin{aligned}Var &\rightarrow Var\ Var \\Var &\rightarrow ter \\Start\ Var &\rightarrow \epsilon\end{aligned}$$

where Var can be any variable, including the Start Variable, $Start\ Var$.

- 1) A CFG in Chomsky Normal Form has derivations of $2n - 1$ steps for generating strings $w \in L(G)$ of length n .
- 2) **Any CFL can be generated by a CFG written in Chomsky Normal Form.**

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

1. Add a new start variable $S' \rightarrow S$
2. Remove ϵ rules of the form $A \rightarrow \epsilon$
 - Remove nullable symbols/rules
3. Remove unit (short) rules of the form $A \rightarrow B$
 - Remove useless symbols/rules
4. Remove long rules of the form $A \rightarrow u_1 u_2 \cdots u_k$
 - Remove useless symbols/rules

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

1. Add a new start variable $S' \rightarrow S$
2. **Remove ϵ rules of the form $A \rightarrow \epsilon$**

For each occurrence of A in the right side of the rule, we add a new rule with the occurrence of A deleted.

E.g.: Consider any rule $B \rightarrow uAvAw$ (u, v, w can be strings of variables and terminals)

Then new rules: $B \rightarrow uAvAw|uvAw|uAvw|uvw$

What if you had a rule such as $B \rightarrow A$? Then we would have needed to add a rule $B \rightarrow \epsilon$ (unless this rule has been already removed) as B is a **nullable variable**.

Repeat this procedure, until all ϵ -rules are removed.

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

1. Add a new start variable $S' \rightarrow S$
2. Remove ϵ rules of the form $A \rightarrow \epsilon$

E.g.:
 $S \rightarrow 0|X0|ZYZ$
 $X \rightarrow Y|\epsilon$
 $Y \rightarrow 1|X$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

Remove ϵ rules of the form $A \rightarrow \epsilon$ (For each occurrence of A in the right side of the rule, add a new rule with the occurrence of A deleted ; Remove nullable variables, Repeat the procedure until all ϵ rules are removed)

E.g.:
 $S \rightarrow 0|X0|ZYZ$
 $X \rightarrow Y|\epsilon$
 $Y \rightarrow 1|X$

To remove $X \rightarrow \epsilon$, we add new rules:
 $S \rightarrow 0|X0|ZYZ$
 $X \rightarrow Y$
 $Y \rightarrow 1|X|\epsilon$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

Remove ϵ rules of the form $A \rightarrow \epsilon$ (For each occurrence of A in the right side of the rule, add a new rule with the occurrence of A deleted ; Remove nullable variables, Repeat the procedure until all ϵ rules are removed)

E.g.:
 $S \rightarrow 0|X0|ZYZ$
 $X \rightarrow Y|\epsilon$
 $Y \rightarrow 1|X$

To remove $X \rightarrow \epsilon$, we add new rules:
 $S \rightarrow 0|X0|ZYZ$
 $X \rightarrow Y$
 $Y \rightarrow 1|X|\epsilon$

To remove $Y \rightarrow \epsilon$, we add:
 $S \rightarrow 0|X0|ZYZ|ZZ$
 $X \rightarrow Y$
 $Y \rightarrow 1|X$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

1. Add a new start variable $S' \rightarrow S$
2. Remove ϵ rules of the form $A \rightarrow \epsilon$
3. **Remove unit rules of the form $A \rightarrow B$**

We **remove the rule $A \rightarrow B$** and **whenever a rule $B \rightarrow u$ appears** (u is a string of terminals and variables), we **add a new rule $A \rightarrow u$** , unless this rule was already removed.

Repeat these steps until all unit rules are removed.

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

Remove unit rules of the form $A \rightarrow B$ (Whenever a rule $B \rightarrow u$ appears, we add a new rule $A \rightarrow u$, unless this rule was already removed. Repeat these steps until all unit rules are removed.)

E.g.:

$$S \rightarrow A|11$$

$$A \rightarrow B|1$$

$$B \rightarrow S|0$$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

Remove unit rules of the form $A \rightarrow B$ (Whenever a rule $B \rightarrow u$ appears, we add a new rule $A \rightarrow u$, unless this rule was already removed. Repeat these steps until all unit rules are removed.)

E.g.:

$$\begin{aligned} S &\rightarrow A|11 \\ A &\rightarrow B|1 \\ B &\rightarrow S|0 \end{aligned}$$

Remove $S \rightarrow A$

$$\begin{aligned} S &\rightarrow 11|B|1 \\ A &\rightarrow B|1 \\ B &\rightarrow S|0 \end{aligned}$$

Remove $A \rightarrow B$

$$\begin{aligned} S &\rightarrow 11|B|1 \\ A &\rightarrow 1|S|0 \\ B &\rightarrow S|0 \end{aligned}$$

Remove $B \rightarrow S$

$$\begin{aligned} S &\rightarrow 11|B|1 \\ A &\rightarrow 1|S|0 \\ B &\rightarrow 0|11|1|B \end{aligned}$$

Remove $B \rightarrow B$

$$\begin{aligned} S &\rightarrow 11|B|1 \\ A &\rightarrow 1|S|0 \\ B &\rightarrow 0|11|1 \end{aligned}$$

Remove $S \rightarrow B$

$$\begin{aligned} S &\rightarrow 11|0|1 \\ A &\rightarrow 1|S|0 \\ B &\rightarrow 0|11|1 \end{aligned}$$

Remove $A \rightarrow S$

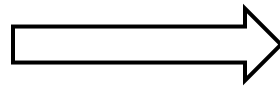
$$\begin{aligned} S &\rightarrow 11|0|1 \\ A &\rightarrow 1|11|0 \\ B &\rightarrow 0|11|1 \end{aligned}$$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

Remove unit rules of the form $A \rightarrow B$ (Whenever a rule $B \rightarrow u$ appears, we add a new rule $A \rightarrow u$, unless this rule was already removed. Repeat these steps until all unit rules are removed.)

$$\begin{aligned} S &\rightarrow A|11 \\ A &\rightarrow B|1 \\ B &\rightarrow S|0 \end{aligned}$$

$$\begin{aligned} S &\rightarrow 11|0|1 \\ A &\rightarrow 1|11|0 \\ B &\rightarrow 0|11|1 \end{aligned}$$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

1. Add a new start variable $S' \rightarrow S$
2. Remove ϵ rules of the form $A \rightarrow \epsilon$
3. Remove unit rules of the form $A \rightarrow B$
- 4. Remove long rules of the form $A \rightarrow u_1 u_2 \cdots u_k$**

Note that each u_i could be a variable or a terminal. We do the following:

- Replace $A \rightarrow u_1 u_2 \cdots u_k$, ($k \geq 3$) with the rules $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2, \dots, A_{k-2} \rightarrow u_{k-1} u_k$
- We replace any terminal u_i in the preceding rules with the new variable U_i and add the rule $U_i \rightarrow u_i$

Chomsky Normal Form

Any CFL can be generated by a CFG written in Chomsky Normal Form.

Proof: The proof is constructive. Suppose we have a CFG G with a set of rules. To convert G into CNF, we do the following:

Add a new start variable $S' \rightarrow S$

Remove ϵ rules of the form $A \rightarrow \epsilon$ (For each occurrence of A in the right side of the rule, add a new rule with the occurrence of A deleted ; Remove nullable variables, Repeat the procedure until all ϵ rules are removed).

Remove unit rules of the form $A \rightarrow B$ (Whenever a rule $B \rightarrow u$ appears, we add a new rule $A \rightarrow u$, unless this rule was already removed. Repeat these steps until all unit rules are removed.)

Remove long rules of the form $A \rightarrow u_1 u_2 \cdots u_k$ (Replace $A \rightarrow u_1 u_2 \cdots u_k$, ($k \geq 3$) with the rules $A \rightarrow u_1 A_1$, $A_1 \rightarrow u_2 A_2, \cdots, A_{k-2} \rightarrow u_{k-1} u_k$; Replace any terminal u_i in the preceding rules with the new variable U_i and add the rule $U_i \rightarrow u_i$).

Chomsky Normal Form

CNF:

$A \rightarrow BC$	$[B, C \text{ are not start variables}]$
$A \rightarrow a$	$[a \text{ is a terminal}]$
$S \rightarrow \epsilon$	$[S \text{ is the Start Variable}]$

Convert the CFG

$$\begin{aligned} S &\rightarrow ASA|aB \\ A &\rightarrow B|S \\ B &\rightarrow b|\epsilon \end{aligned}$$

to CNF.

1. Add a new start variable

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow ASA|aB \\ A &\rightarrow B|S \\ B &\rightarrow b|\epsilon \end{aligned}$$

2a. Remove ϵ rules ($B \rightarrow \epsilon$)

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow ASA|aB|a \\ A &\rightarrow B|S|\epsilon \\ B &\rightarrow b \end{aligned}$$

2b. Remove ϵ rules ($A \rightarrow \epsilon$)

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow ASA|aB|a|AS|SA|S \\ A &\rightarrow B|S \\ B &\rightarrow b \end{aligned}$$

Chomsky Normal Form

CNF:

$A \rightarrow BC$	$[B, C \text{ are not start variables}]$
$A \rightarrow a$	$[a \text{ is a terminal}]$
$S \rightarrow \epsilon$	$[S \text{ is the Start Variable}]$

Convert the CFG

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\epsilon$$

to CNF.

3a. Remove $S \rightarrow S$

$$S' \rightarrow S$$

$$S \rightarrow ASA|aB|a|AS|SA$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$

3b. Remove $S' \rightarrow S$

$$S' \rightarrow ASA|aB|a|AS|SA$$

$$S \rightarrow ASA|aB|a|AS|SA$$

$$A \rightarrow B|S$$

$$B \rightarrow b$$

3c. Remove $A \rightarrow B$

$$S' \rightarrow ASA|aB|a|AS|SA$$

$$S \rightarrow ASA|aB|a|AS|SA$$

$$A \rightarrow S|b$$

$$B \rightarrow b$$

3d. Remove $A \rightarrow S$

$$S' \rightarrow ASA|aB|a|AS|SA$$

$$S \rightarrow ASA|aB|a|AS|SA$$

$$A \rightarrow b|ASA|aB|a|AS|SA$$

$$B \rightarrow b$$

Chomsky Normal Form

CNF:

$A \rightarrow BC$	$[B, C \text{ are not start variables}]$
$A \rightarrow a$	$[a \text{ is a terminal}]$
$S \rightarrow \epsilon$	$[S \text{ is the Start Variable}]$

Convert the CFG

$$S \rightarrow ASA|aB$$

$$A \rightarrow B|S$$

$$B \rightarrow b|\epsilon$$

to CNF.

3d. Remove $A \rightarrow S$

$$S' \rightarrow ASA|aB|a|AS|SA$$

$$S \rightarrow ASA|aB|a|AS|SA$$

$$A \rightarrow b|ASA|aB|a|AS|SA$$

$$B \rightarrow b$$

4a. Remove long rules

$$S' \rightarrow ASA|aB|a|AS|SA$$

$$S \rightarrow ASA|aB|a|AS|SA$$

$$A \rightarrow b|ASA|aB|a|AS|SA$$

$$B \rightarrow b$$

4b. Remove long rules

$$S' \rightarrow AU|aB|a|AS|SA$$

$$S \rightarrow AU|aB|a|AS|SA$$

$$A \rightarrow b|AU|aB|a|AS|SA$$

$$U \rightarrow SA$$

$$B \rightarrow b$$

4c. Remove long rules

$$S' \rightarrow AU|VB|a|AS|SA$$

$$S \rightarrow AU|VB|a|AS|SA$$

$$A \rightarrow b|AU|VB|a|AS|SA$$

$$U \rightarrow SA$$

$$V \rightarrow a$$

$$B \rightarrow b$$

There are other rules of the form: $\text{Var} \rightarrow ASA$

Chomsky Normal Form

CNF: $A \rightarrow BC$ [B, C are not start variables]
 $A \rightarrow a$ [a is a terminal]
 $S \rightarrow \epsilon$ [S is the Start Variable]

Convert the CFG

$$\begin{aligned} S &\rightarrow ASA|aB \\ A &\rightarrow B|S \\ B &\rightarrow b|\epsilon \end{aligned}$$

to CNF.

$$\begin{aligned} S' &\rightarrow AU|VB|a|AS|SA \\ S &\rightarrow AU|VB|a|AS|SA \\ A &\rightarrow b|AU|VB|a|AS|SA \\ U &\rightarrow SA \\ V &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

Pushdown Automata

- For regular languages we had
 - Designed Finite automata (DFA, NFA) that recognize the strings by the language. Helped us decide whether a given string ω belongs to the language.

Pushdown Automata

- For regular languages we had
 - Designed Finite automata (DFA, NFA) that recognize the strings by the language. Helped us decide whether a given string ω belongs to the language.
 - Developed regular expressions/linear grammar that can generate all the strings in the language.

Pushdown Automata

- For regular languages we had
 - Designed Finite automata (DFA, NFA) that recognize the strings by the language. Helped us decide whether a given string ω belongs to the language.
 - Developed regular expressions/linear grammar that can generate all the strings in the language.
- For context free languages,
 - Context Free Grammars generate all the strings in the language

Pushdown Automata

- For regular languages we had
 - Designed Finite automata (DFA, NFA) that recognize the strings by the language. Helped us decide whether a given string ω belongs to the language.
 - Developed regular expressions/linear grammar that can generate all the strings in the language.
- For context free languages,
 - Context Free Grammars generate all the strings in the language
 - Can we build an automata that recognizes **exactly** context free languages?

Pushdown Automata

- For regular languages we had
 - Designed Finite automata (DFA, NFA) that recognize the strings by the language. Helped us decide whether a given string ω belongs to the language.
 - Developed regular expressions/linear grammar that can generate all the strings in the language.
- For context free languages,
 - Context Free Grammars generate all the strings in the language
 - Can we build an automata that recognizes **exactly** context free languages?
- **Finite Automaton model recognizes ALL regular languages**
- Any automata that recognizes **ALL** context free languages will need unbounded memory.

Pushdown Automata

- Finite Automaton model recognizes ALL regular languages
- Any automata that recognizes **ALL** context free languages will need unbounded memory.

Intuition to build an Automata for CFL

- It should be some **Finite State Machine** that has access to a memory device with infinite memory, i.e.

Automata for CFL = FSM + Memory device

- **FSM may choose to ignore the memory device** completely in which case it behaves like a DFA/NFA.
- FSM makes use of the Memory device to recognize “non-Regular” CFLs.

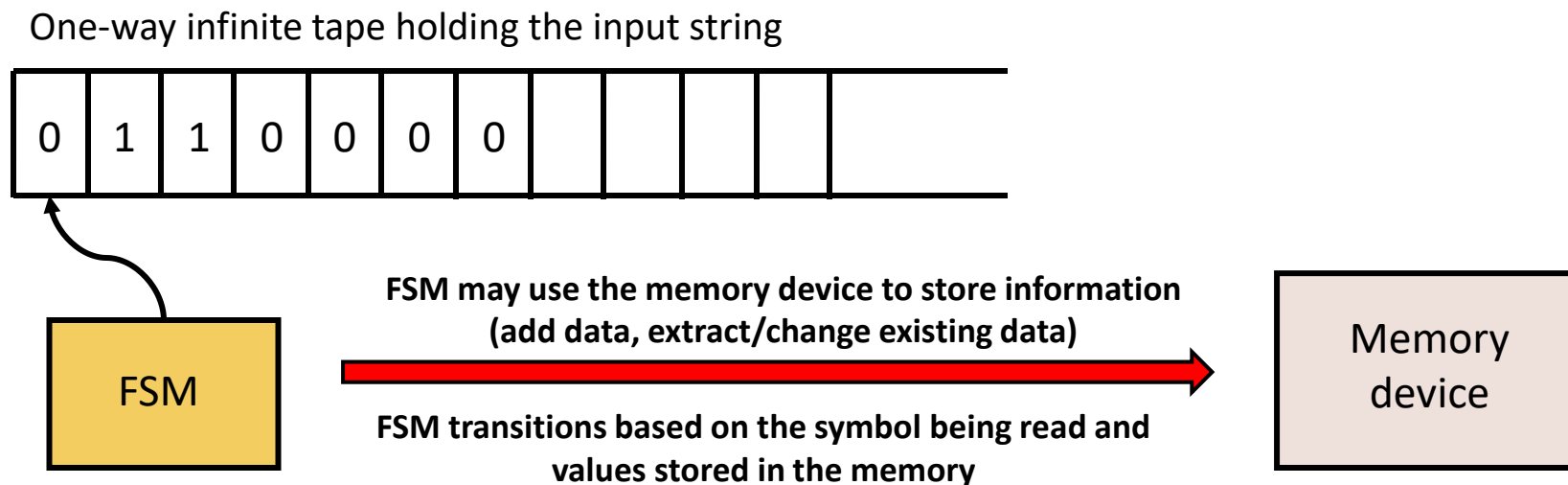
E.g.: $\{0^n 1^n, n \in \mathbb{N}\}$

Pushdown Automata

Intuition to build an Automata for CFL

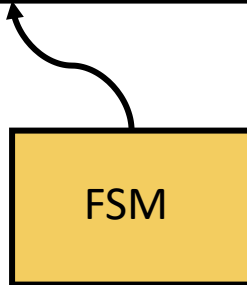
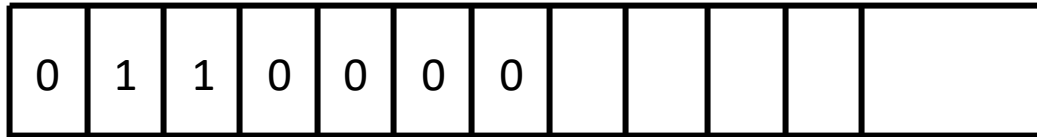
- **Automata for CFL = FSM + Memory device**
- **FSM may choose to ignore the memory device** completely in which case it behaves like a DFA/NFA.
- FSM makes use of the Memory device to recognize “non-Regular” CFLs.

E.g.: $\{0^n 1^n, n \in \mathbb{N}\}$



Pushdown Automata

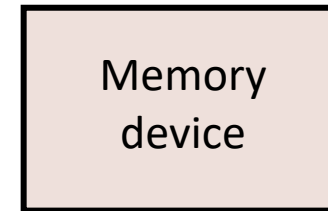
One-way infinite tape holding the input string



FSM may use the memory device to store information
(add data, extract/change existing data)



FSM transitions based on the symbol being read and
values stored in the memory



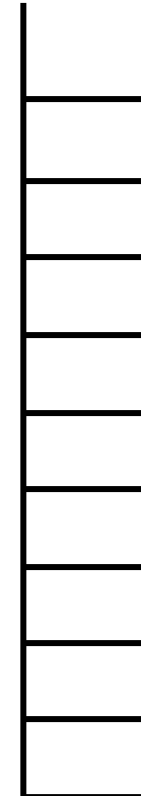
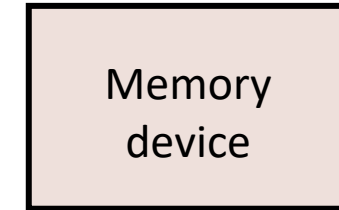
The memory device

- Simple memory device with unbounded memory.

Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, new elements can be added to the Stack (**PUSH**).
- At any stage, the element at the **top** of the STACK can be read by removing it from the stack (**POP**).



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.

PUSH

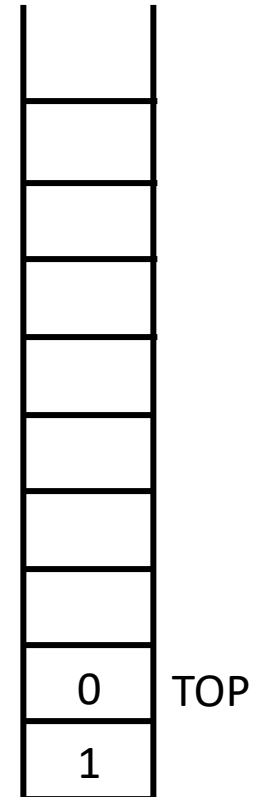
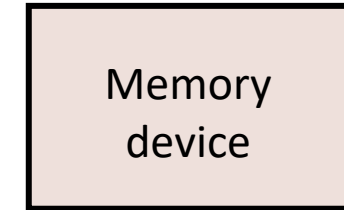
- New symbols can be **pushed** in to the STACK.

E.g: PUSH 1

- The Top of the STACK now covers the old stack top, i.e.

$TOP = TOP + 1$

- The size of the stack keeps growing.



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.

PUSH

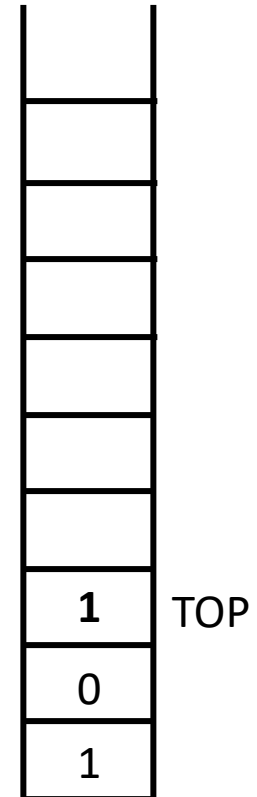
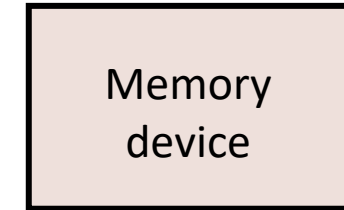
- New symbols can be **pushed** in to the STACK.

E.g: PUSH 1

- The Top of the STACK now covers the old stack top, i.e.

$TOP = TOP + 1$

- The size of the stack keeps growing.



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.

PUSH

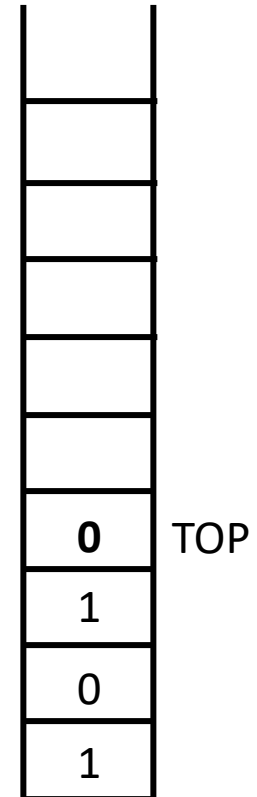
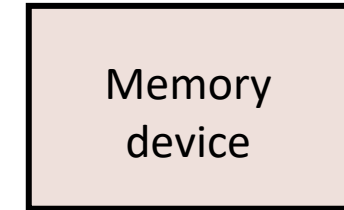
- New symbols can be **pushed** in to the STACK.

E.g: **PUSH 0**

- The Top of the STACK now covers the old stack top, i.e.

$$\text{TOP} = \text{TOP} + 1$$

- The size of the stack keeps growing.



Pushdown Automata

The memory device

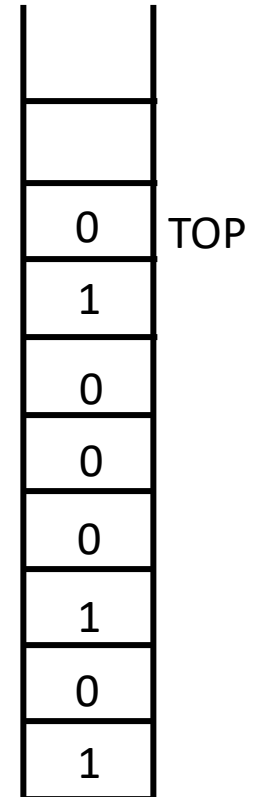
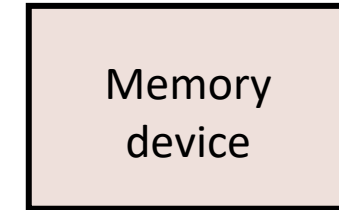
- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.

PUSH

- New symbols can be **pushed** in to the STACK.
- The Top of the STACK now covers the old stack top, i.e.

$$\text{TOP} = \text{TOP} + 1$$

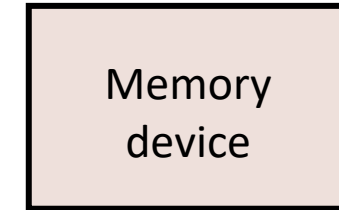
- The size of the stack keeps growing.



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.



POP

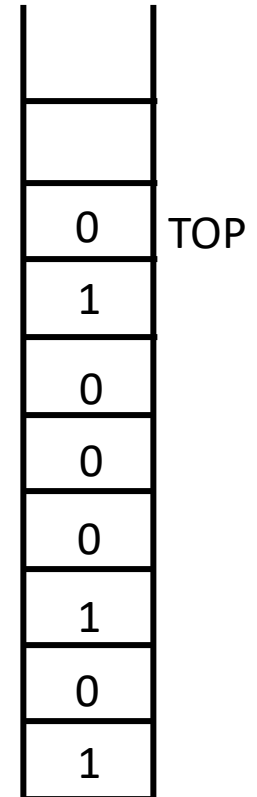
- The element from the TOP of the stack can be **popped** out

E.g.: **POP 0**

- The Top of the STACK moves to the element below.

$$\text{TOP} = \text{TOP} - 1$$

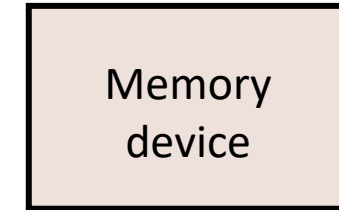
- Successive **POP** operations shrink the stack size. Elements can be popped until EMPTY.
- **Last In First Out (LIFO)**: The last element that was pushed is the first to be popped out



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.



POP

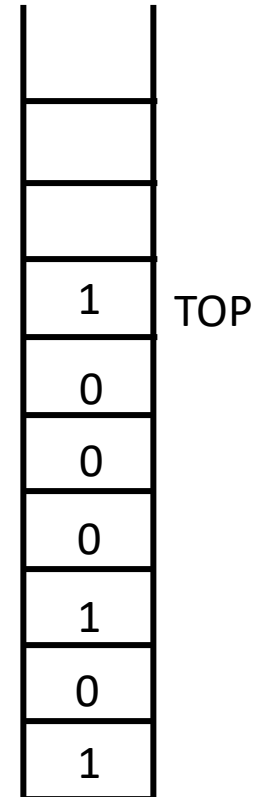
- The element from the TOP of the stack can be **popped** out

E.g.: **POP 0**

- The Top of the STACK moves to the element below.

$$\text{TOP} = \text{TOP} - 1$$

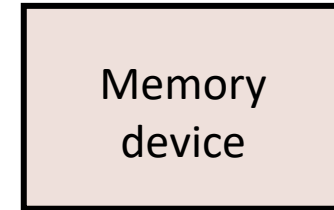
- Successive **POP** operations shrink the stack size. Elements can be popped until EMPTY.
- **Last In First Out (LIFO)**: The last element that was pushed is the first to be popped out



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.



POP

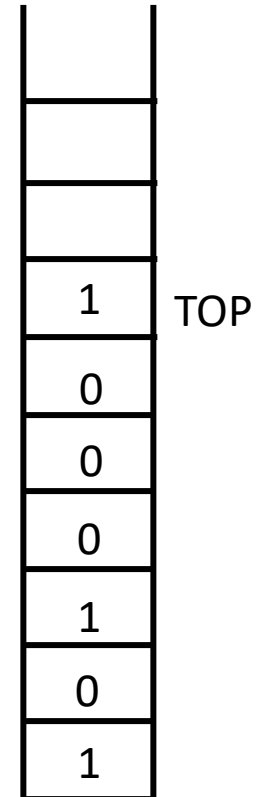
- The element from the TOP of the stack can be **popped** out

E.g.: **POP 1**

- The Top of the STACK moves to the element below.

$$\text{TOP} = \text{TOP} - 1$$

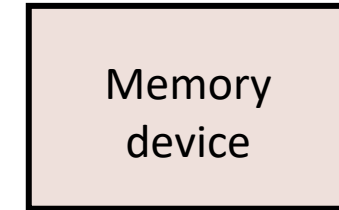
- Successive **POP** operations shrink the stack size. Elements can be popped until EMPTY.
- **Last In First Out (LIFO)**: The last element that was pushed is the first to be popped out



Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- At any stage, elements can be pushed or popped.



POP

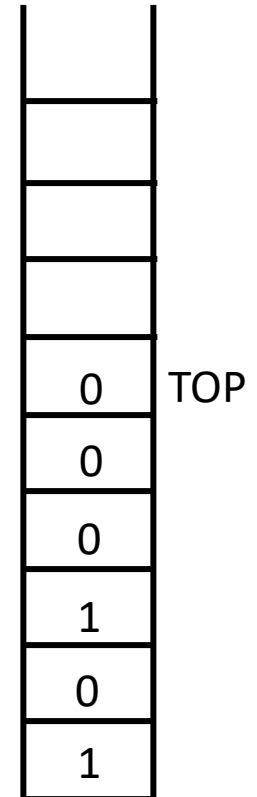
- The element from the TOP of the stack can be **popped** out

E.g.: **POP 1**

- The Top of the STACK moves to the element below.

$$\text{TOP} = \text{TOP} - 1$$

- Successive **POP** operations shrink the stack size. Elements can be popped until EMPTY.
- **Last In First Out (LIFO)**: The last element that was pushed is the first to be popped out



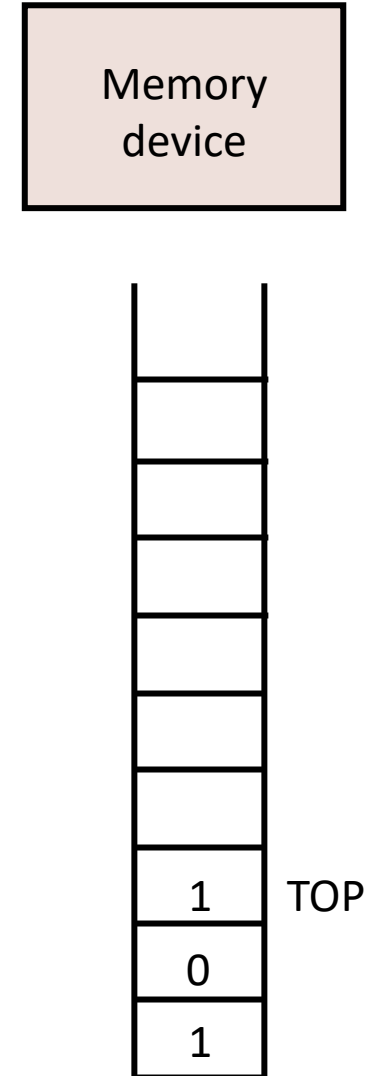
Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- Last In First Out (**LIFO**)

POP

- The element from the TOP of the stack can be **popped** out.
 - $TOP = TOP - 1$
 - Elements can be popped until STACK is EMPTY.
-
- How would you know that the STACK is EMPTY?



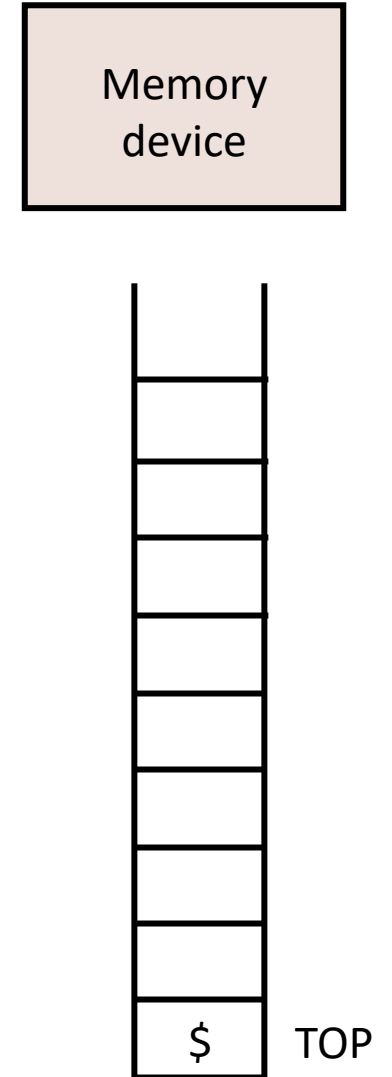
Pushdown Automata

The memory device

- Simple memory device with unbounded memory.
- Consider a **STACK**
- Last In First Out (**LIFO**)

POP

- The element from the TOP of the stack can be **popped** out.
 - $TOP = TOP - 1$
 - Elements can be popped until STACK is EMPTY.
-
- How would you know that the STACK is EMPTY?
 - There is generally some special symbol (say \$) that demarcates the bottom of the STACK.
 - This element is Pushed at the very beginning. Whenever the popped element = \$, the STACK is EMPTY.



Pushdown Automata

Memory device of PDA: STACK

- STACK is a **LIFO** data structure of unbounded memory
- Only the TOP element can be read from the STACK.
- The bottom of the STACK contains a special symbol (\$)
- Characterized by two operations:

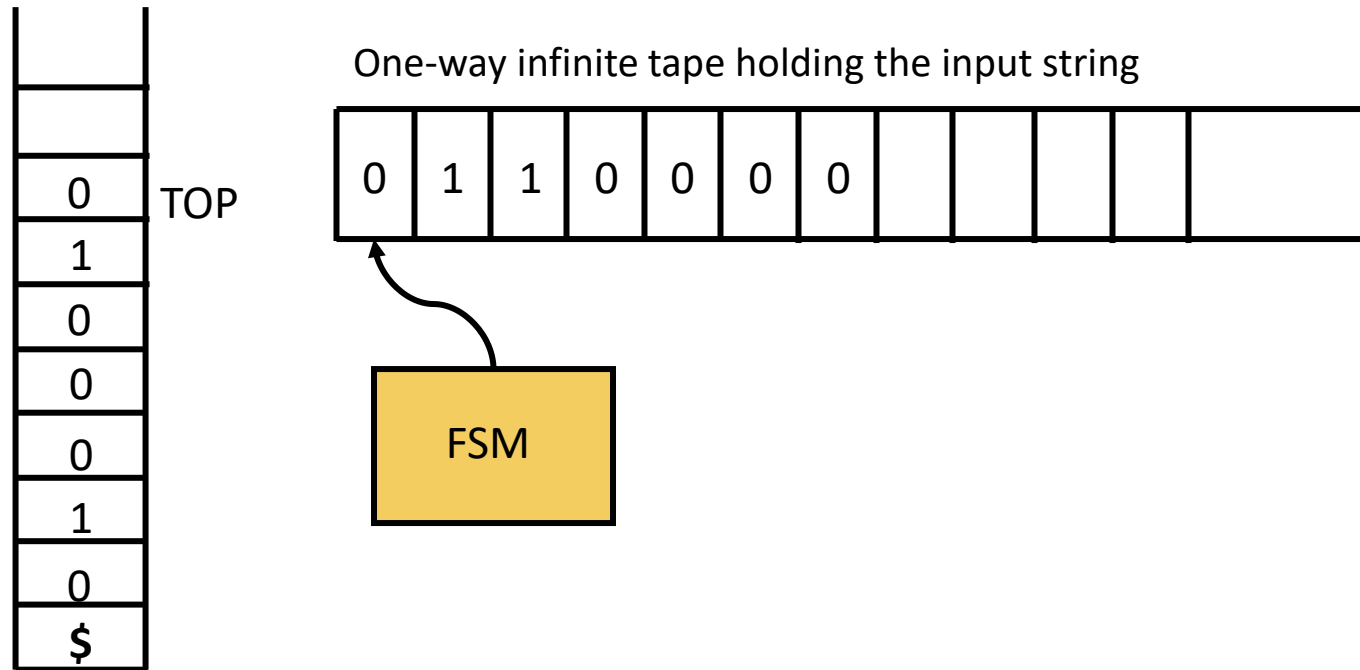
PUSH

- New symbols can be **pushed** in to the STACK.
- $TOP = TOP + 1$

POP

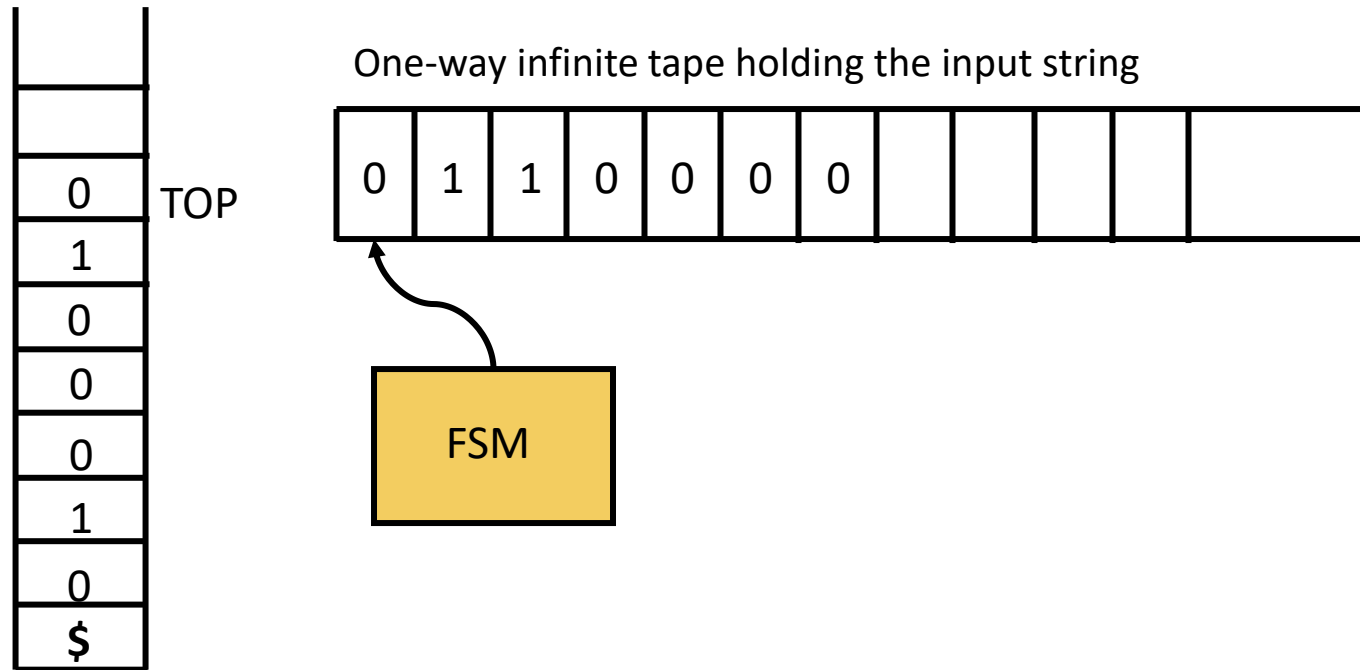
- The element from the TOP of the stack can be **popped** out.
- $TOP = TOP - 1$
- Elements can be popped until STACK is EMPTY.

Pushdown Automata



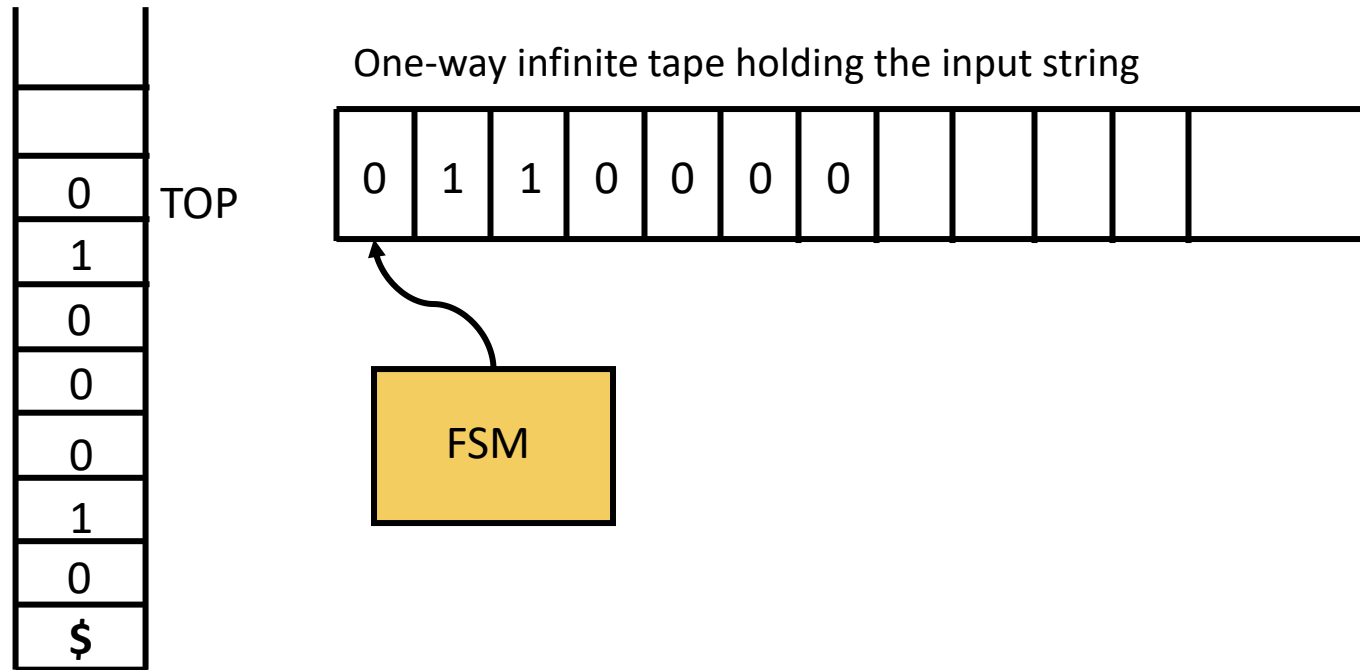
- A Pushdown Automata (PDA) is a finite automaton that has access to a stack.
- The FSM:

Pushdown Automata



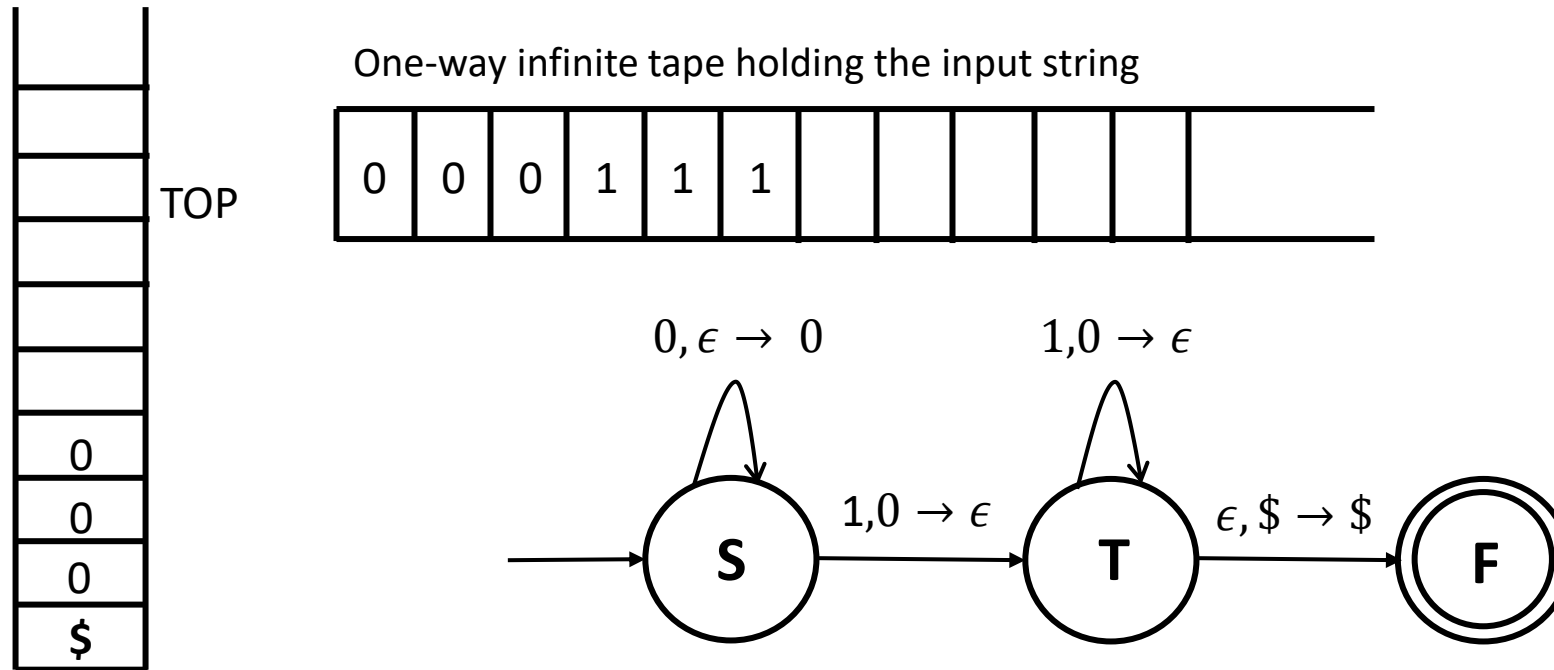
- A Pushdown Automata (PDA) is a finite automaton that has access to a stack.
- The FSM:
 - Transitions based on the Input symbol and the element at the top of the stack (e.g.: If I/P symbol = 0 & 0 is popped, transition from i to j)

Pushdown Automata



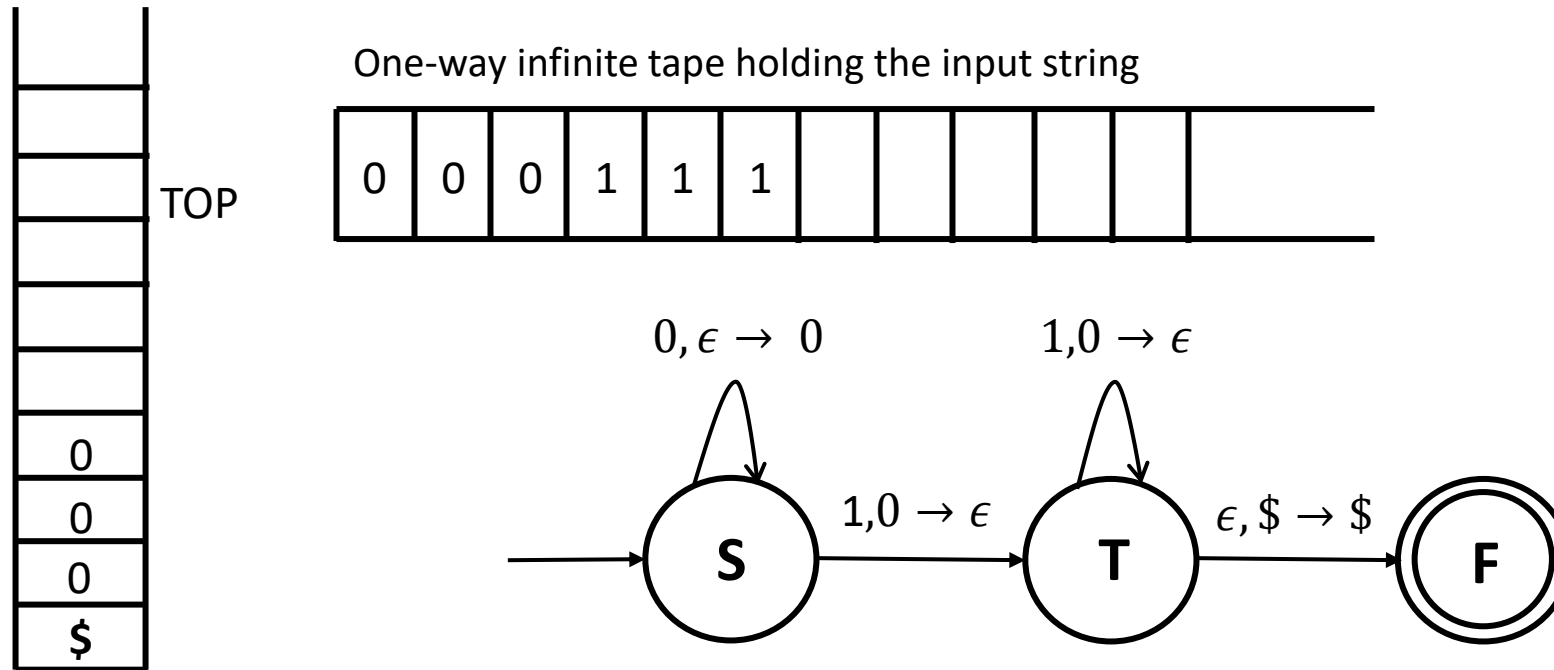
- A Pushdown Automata (PDA) is a finite automaton that has access to a stack.
- The FSM:
 - Transitions based on the Input symbol and the element at the top of the stack (e.g.: If I/P symbol = 0 & POP 0, transition from i to j)
 - Pushes new elements into the Stack (e.g.: If I/P symbol = 0, PUSH 0, transition from i to j).

Pushdown Automata



- A Pushdown Automata (PDA) is a finite automaton that has access to a stack.
- The FSM:
 - Transitions based on the Input symbol and the element at the top of the stack
 - Pops the element at the top of the Stack.
 - Pushes new elements into the Stack.

Pushdown Automata

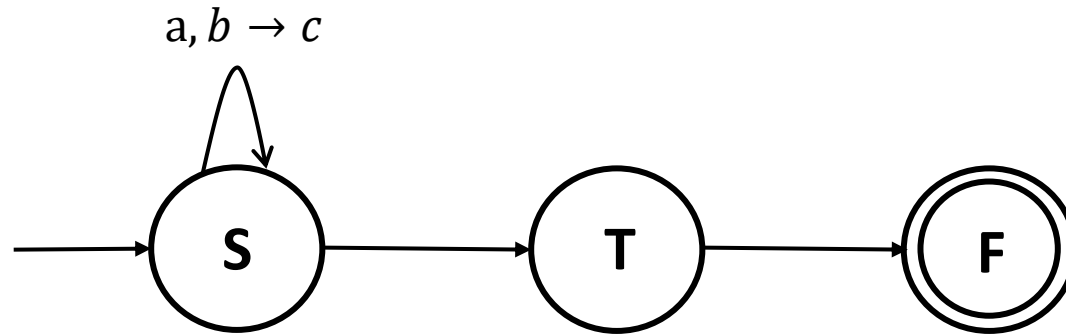


PDA's are **non-deterministic**.

- Missing transitions
- ϵ -transitions
- Multiple transitions/input symbol possible

Pushdown Automata

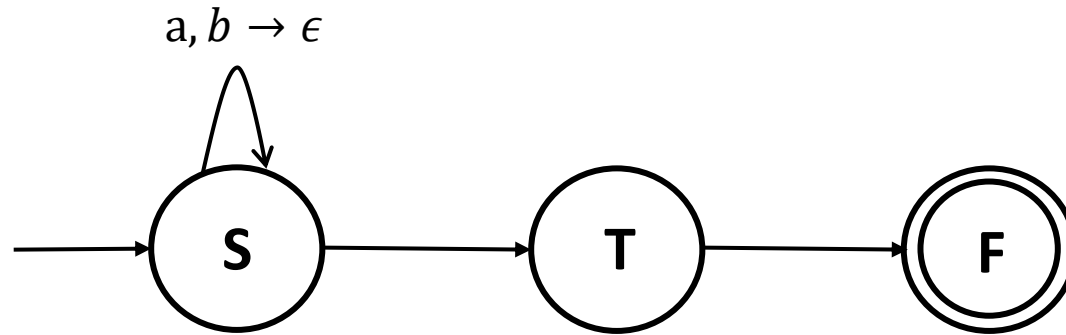
- How to represent a transition in a PDA?



If input symbol = a , Stack top = b (if b is popped), Push c onto the Stack and remain in S

Pushdown Automata

- How to represent a transition in a PDA?

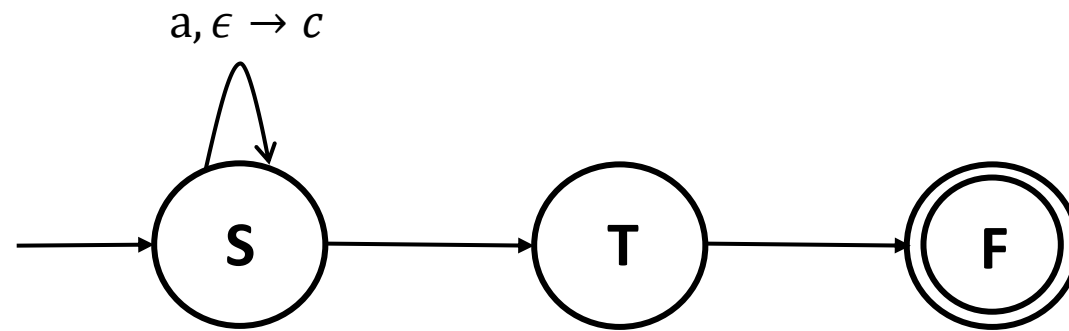


If input symbol = a , and b is popped, remain in S .

(If the symbol read is a and the element at the Stack TOP = b , then remain in S)

Pushdown Automata

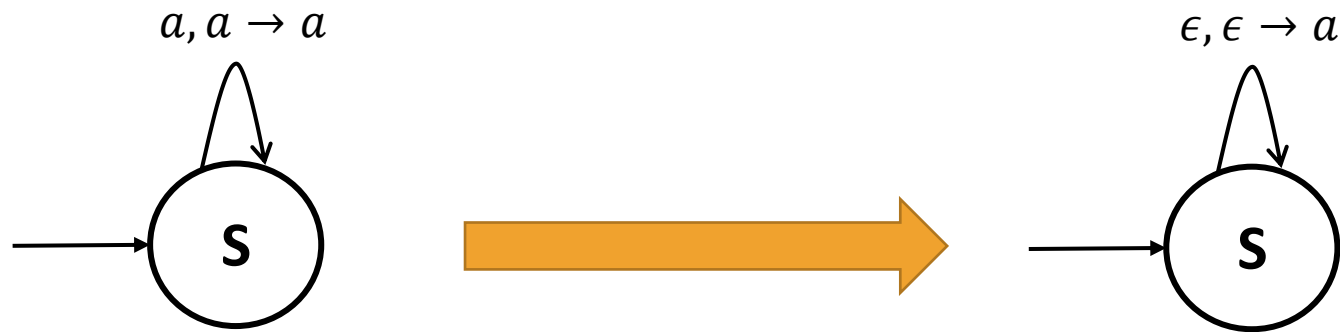
- How to represent a transition in a PDA?



If input symbol = a , then Push c

Pushdown Automata

- How to represent a transition in a PDA?

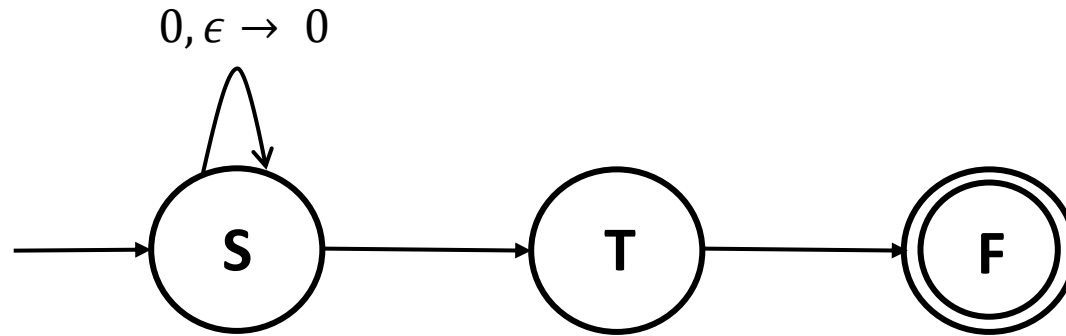


- (i) If input symbol = a , and a is popped, then Push a and remain in S .
- (ii) Push a on to the stack and remain in S .

Through Steps (i) and (ii), the PDA pushes a onto the stack if it reads a on the input tape and the element at the stack top = a .

Pushdown Automata

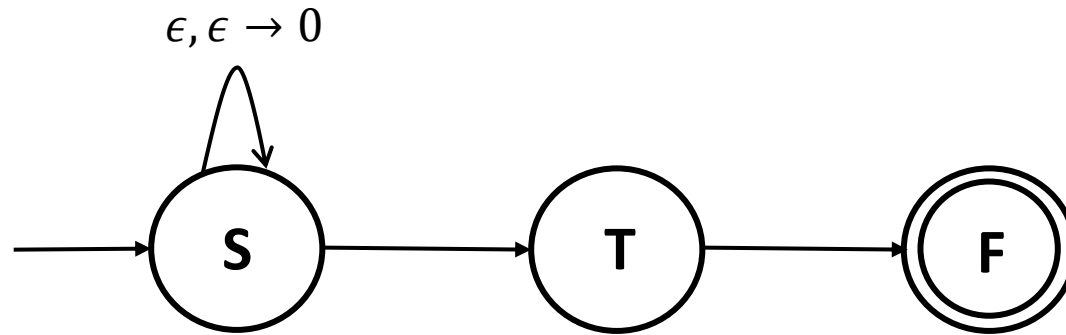
- How to represent a transition in a PDA?



If input symbol = 0, Push 0 onto the Stack irrespective of the element at the top of the stack

Pushdown Automata

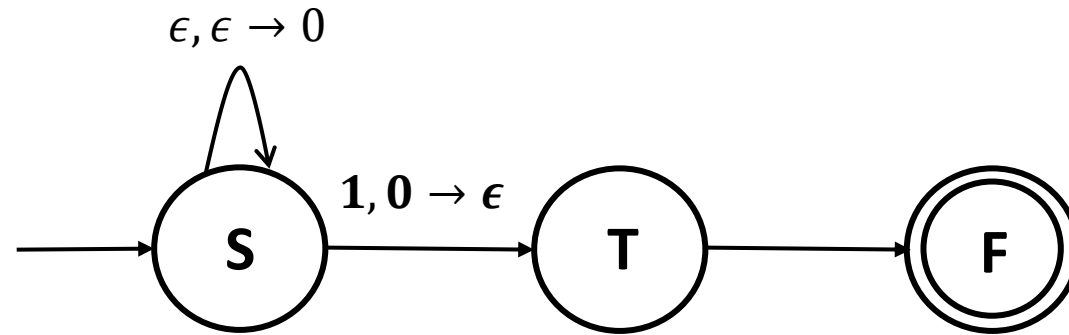
- How to represent a transition in a PDA?



Without reading the input symbol and the Stack top, Push 0 onto the Stack

Pushdown Automata

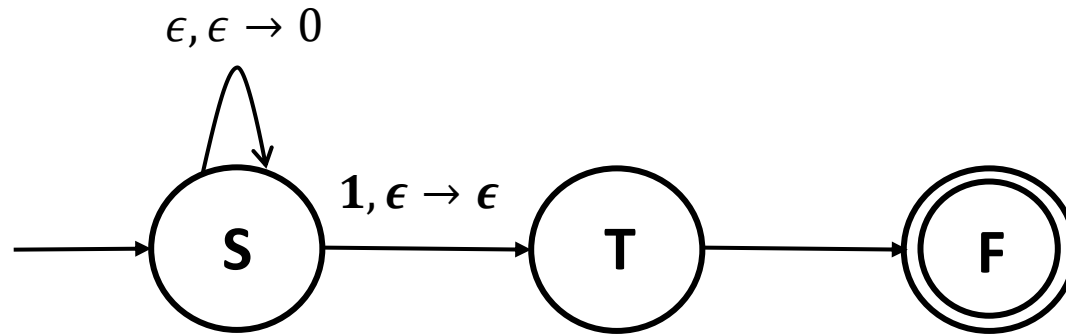
- How to represent a transition in a PDA?



If the input symbol is 1, and the element 0 is popped (**Pop 0**), then transition from S to T

Pushdown Automata

- How to represent a transition in a PDA?

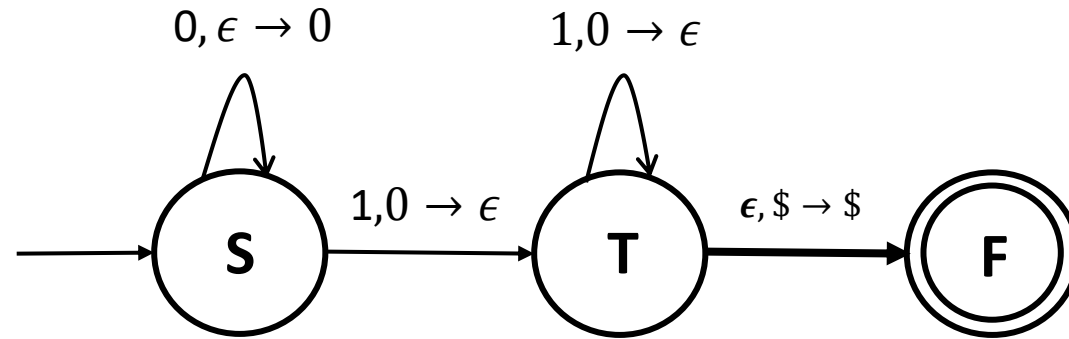


If the input symbol is 1, transition to T by ignoring the stack completely.

If this happens at every step of the execution of the PDA, then it is as powerful as an NFA.

Pushdown Automata

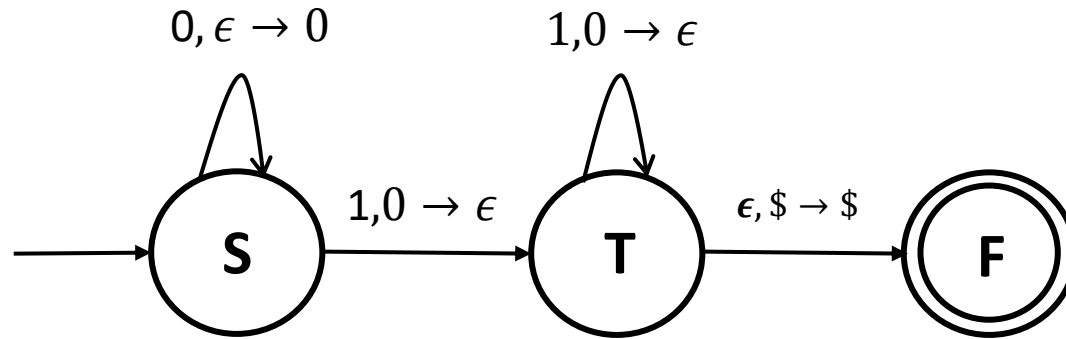
- How to represent a transition in a PDA?



Empty stack: If ϵ is popped, push the ϵ back onto the stack and transition to F from T , without reading the input

Pushdown Automata

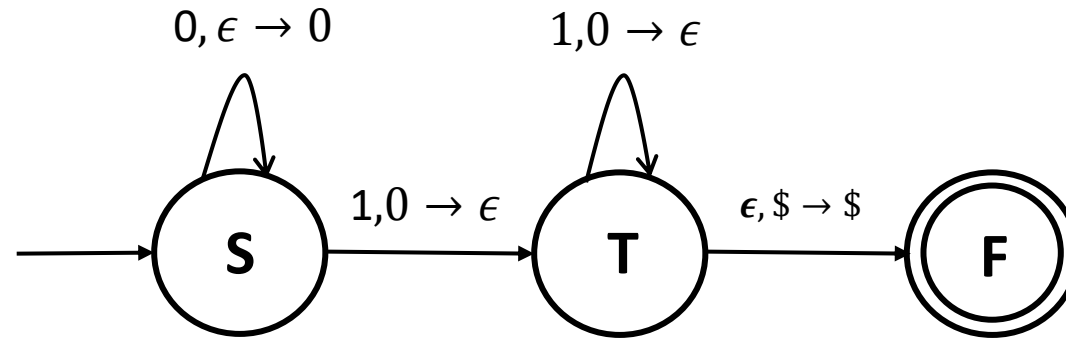
- How to represent a transition in a PDA?



What is the language recognized by this PDA?

Pushdown Automata

- How to represent a transition in a PDA?

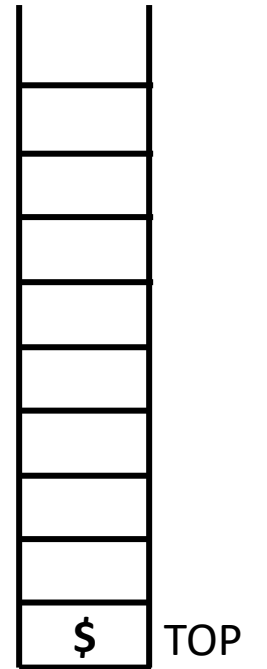
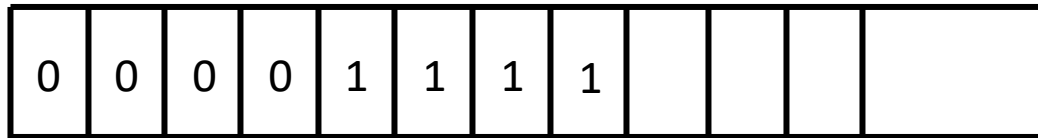
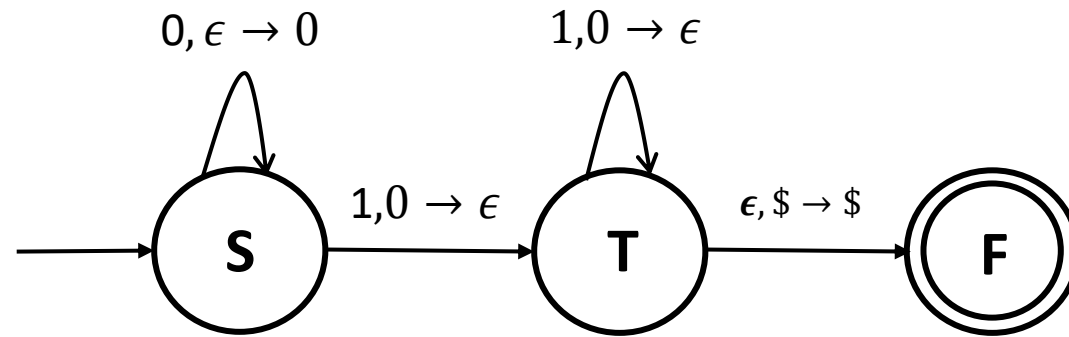


What is the language recognized by this PDA?

Verify that it is $L = \{0^n 1^n, n \geq 1\}$

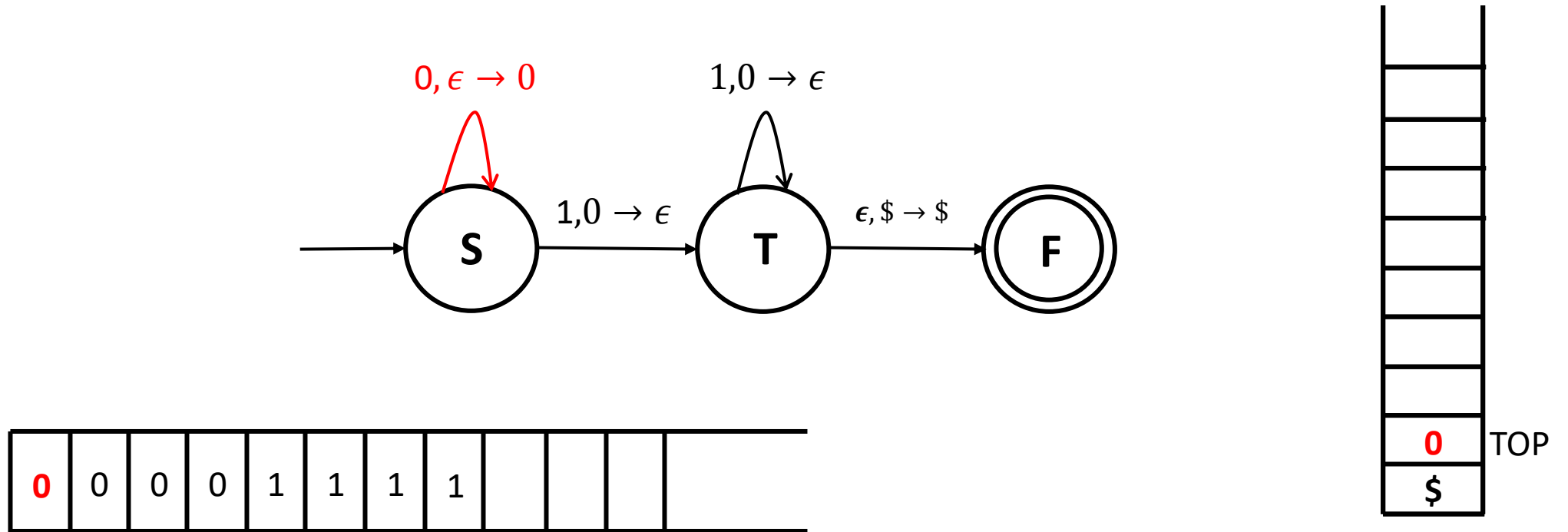
Pushdown Automata

What is the language recognized by this PDA?



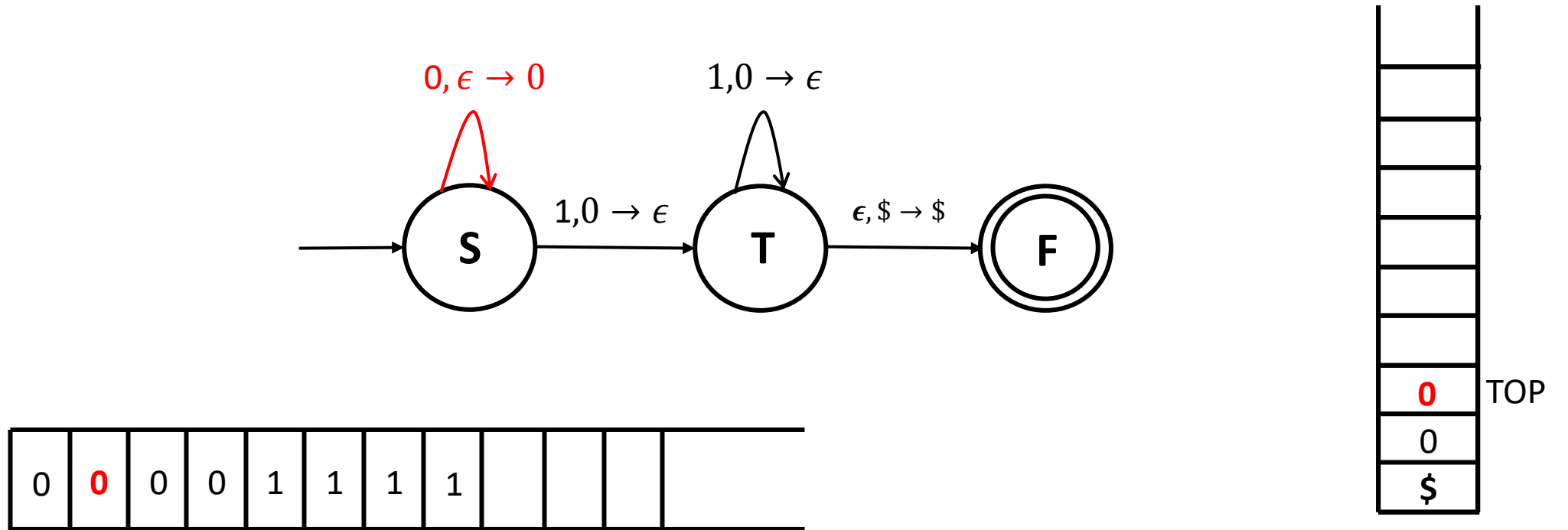
Pushdown Automata

What is the language recognized by this PDA?



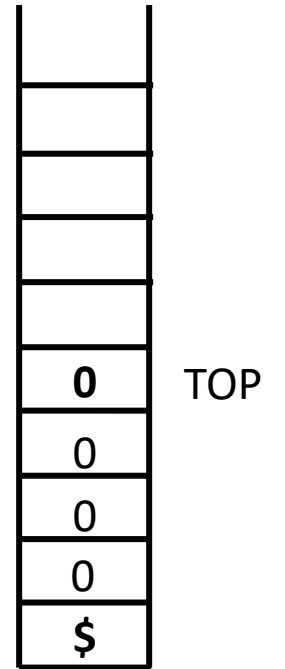
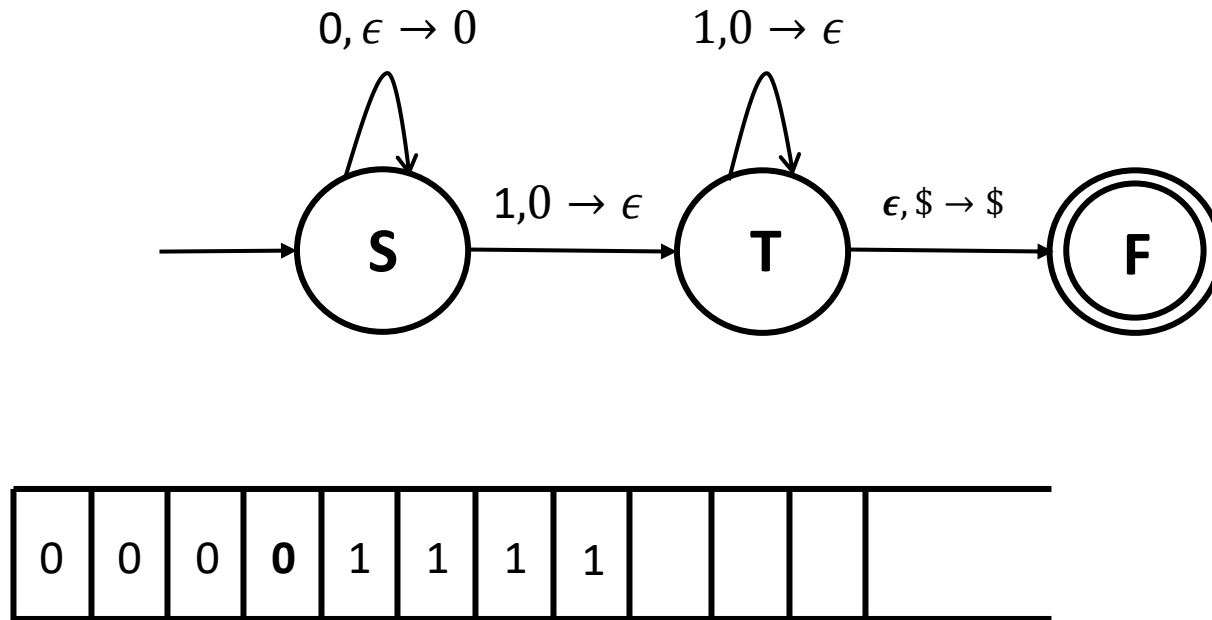
Pushdown Automata

What is the language recognized by this PDA?



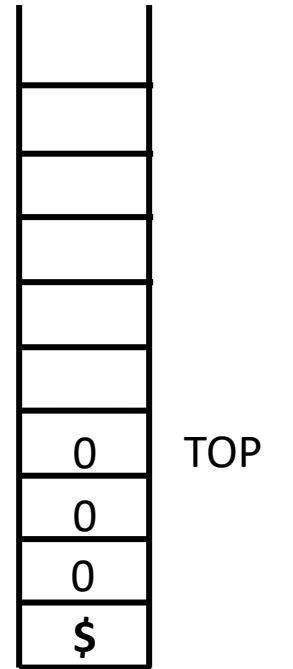
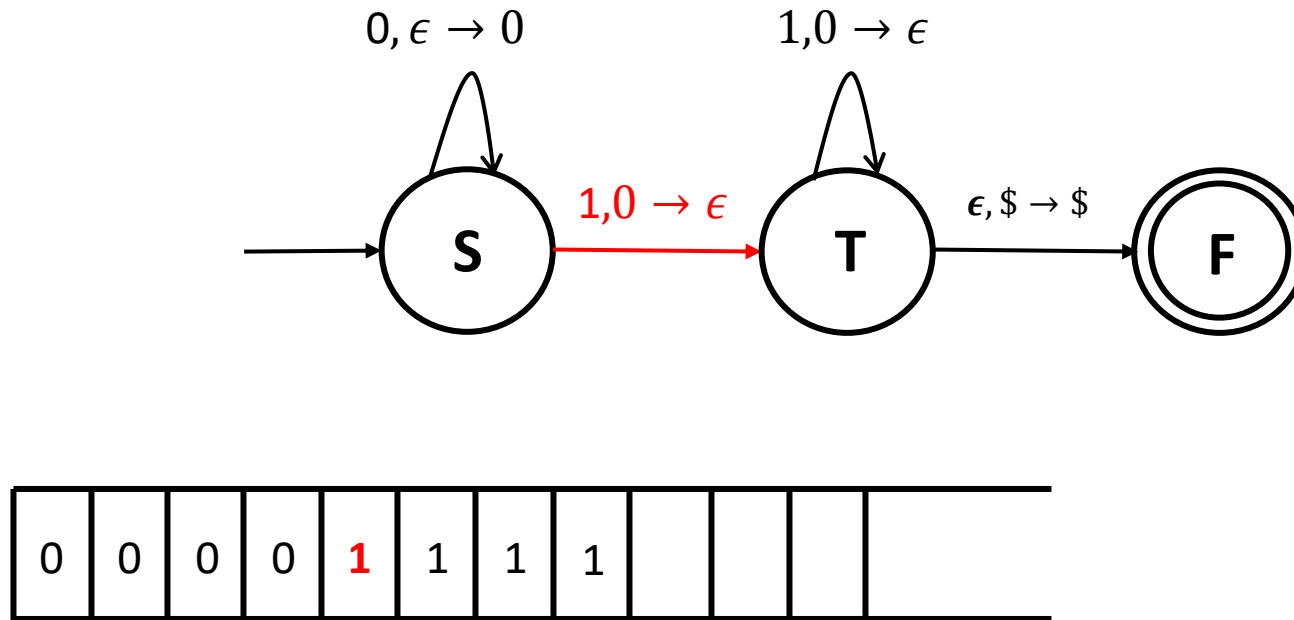
Pushdown Automata

What is the language recognized by this PDA?



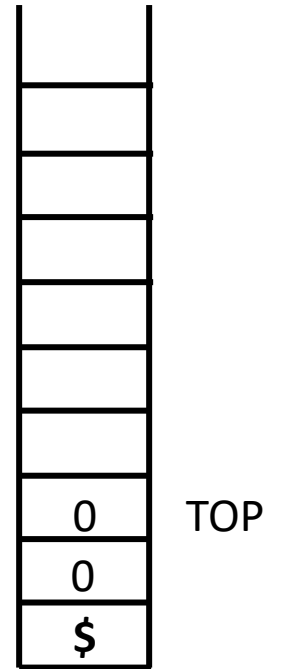
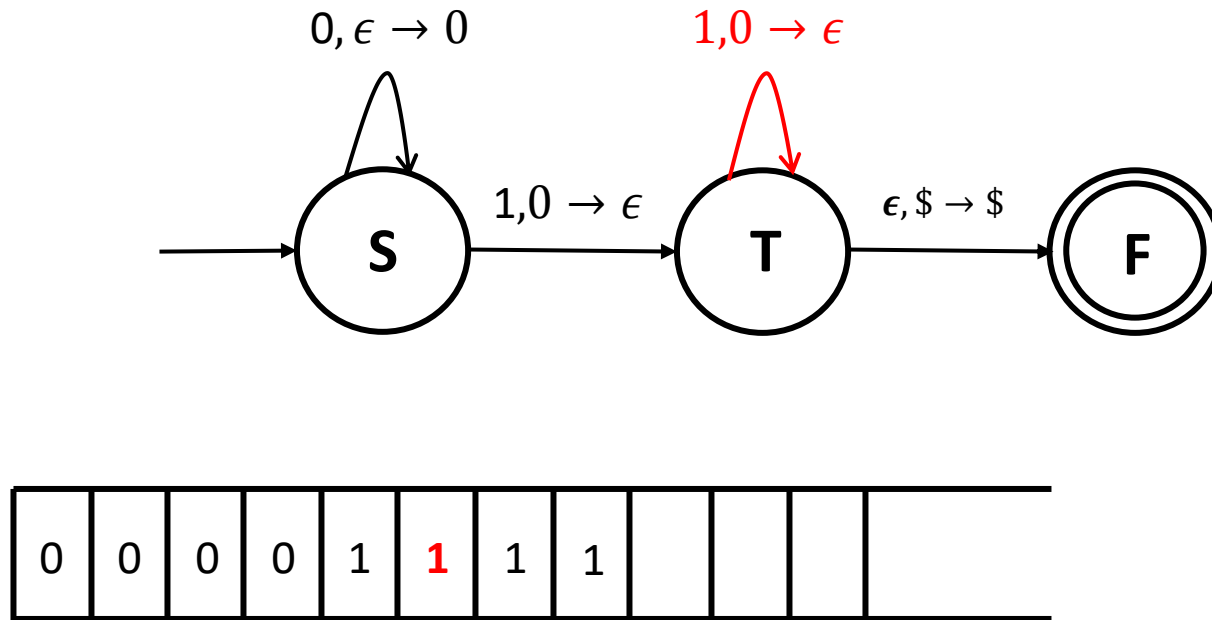
Pushdown Automata

What is the language recognized by this PDA?



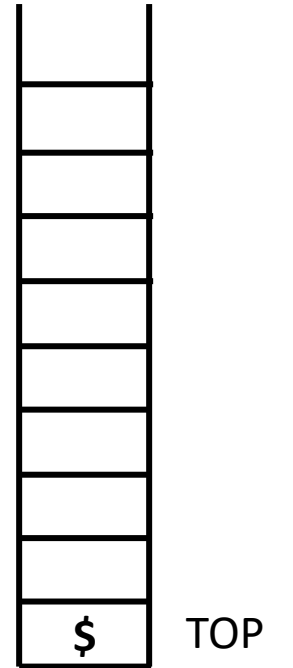
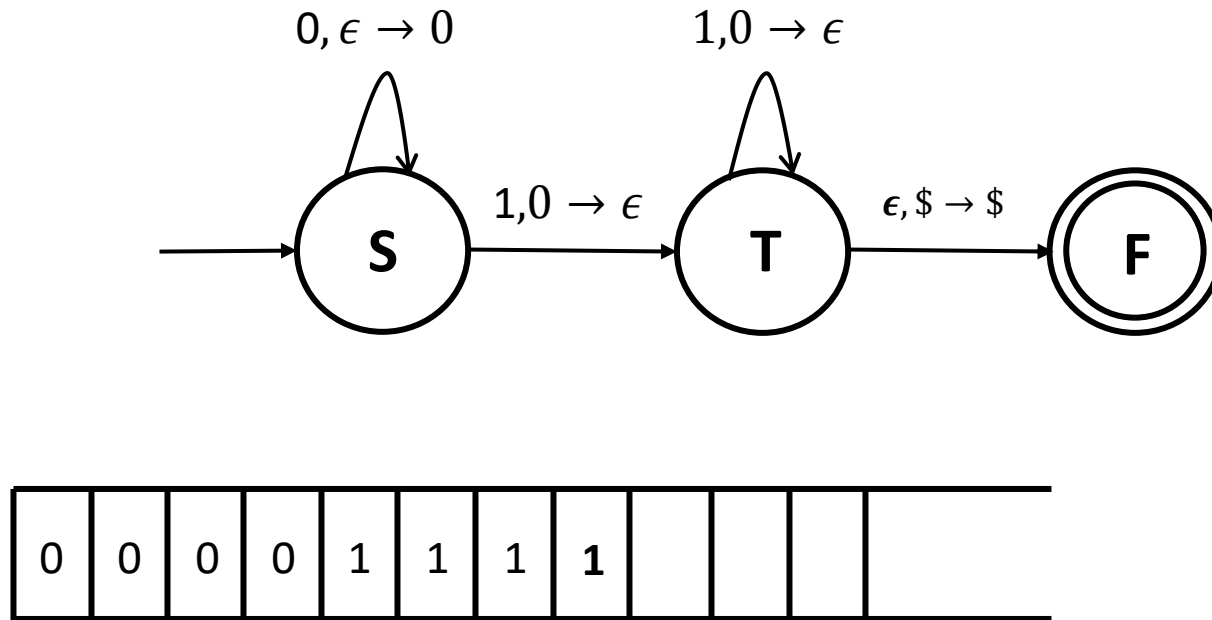
Pushdown Automata

What is the language recognized by this PDA?



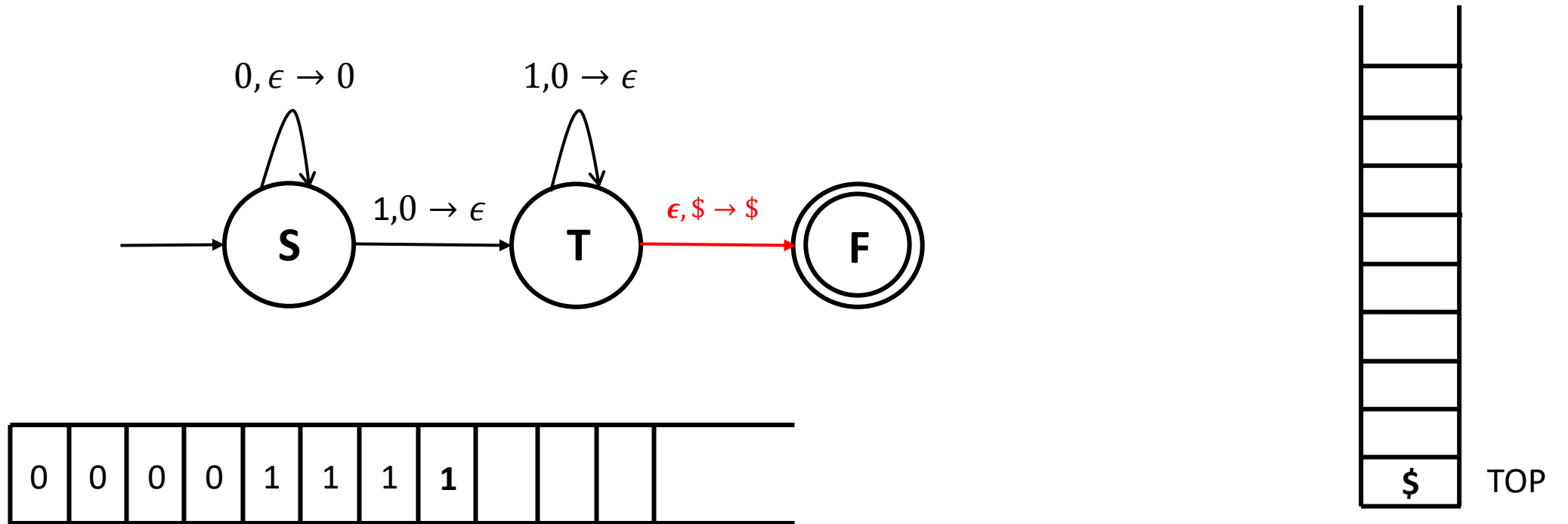
Pushdown Automata

What is the language recognized by this PDA?



Pushdown Automata

What is the language recognized by this PDA?



The language recognized by the PDA: $L = \{0^n 1^n, n \geq 1\}$

Thank You!